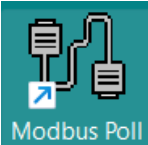


Wk0600 -- Modbus and Modbus TCP	
Std01.รหัส _____	ชื่อ-สกุล _____
Std01.รหัส _____	ชื่อ-สกุล _____

การทดลอง

1. Modbus TCP Test System

 192.168.1.X	 192.168.1.Y
 https://www.modbustools.com/index.html	
Client	Server

2. ติดตั้ง Arduino ESP32 ModbusIP Library

- 2.1 from https://github.com/JoaoCampos89/ModbusIP_ESP32/tree/master >> Download https://github.com/JoaoCampos89/ModbusIP_ESP32/archive/refs/heads/master.zip
- 2.2 Download <https://github.com/andresarmento/modbus-arduino/archive/refs/heads/master.zip>
- 2.3 Sketch → Inc Library → Add Zip ----> Select file “ModbusIP_ESP32-master.zip”
- 2.4 Unzip file “modbus-arduino-master.Zip” copy Modbus libraries to
C:\Users\XXXX\Documents\Arduino\libraries

3. Blink Test ESP32: DOIT ESP32 DevKit Board

4. Modbus TCP Server Mode

- | | |
|---------------------------------|-----------------------------------|
| 4.1 Read/Write Coils | PC: Modbus Poll ← → ESP32(Server) |
| 4.2 Read Holding Register | PC: Modbus Poll ← ESP32(Server) |
| 4.3 Read/Write Holding Register | PC: Modbus Poll ← → ESP32(Server) |
| 4.4 Read Discreet Input | PC: Modbus Poll ← ESP32(Server) |
| 4.5 Read Input Register | PC: Modbus Poll ← ESP32(Server) |

5. Modbus TCP Client Mode

6. การจองหน่วยความจำ การอ่าน และการเขียนค่า ทำได้ด้วยคำสั่ง ดังนี้

<pre> 1 // allocate memory in setup 2 mb.addHreg(Addr0, Data); // allocate Holding register + Store Data 3 mb.addIreg(Addr1); // allocate Inputs register 4 mb.addCoil(Addr2); // allocate Coil input/output 5 md.addIsts(Addr3); // allocate Digital inputs 6 7 // Write data to memory for Client Read 8 mb.Hreg(Addr0, Value); // Write Holding register 9 mb.Iregs(Addr1, Value); // Write Inputs register 10 mb.Coils(Addr2, 1); // Write Coil input/output 11 mb.Ists(Addr3, 1); // Write Digital inputs 12 13 // From Client Write -> Read data from memory 14 int16_t holdingRegValue = mb.Hreg(Addr0); // Read Holding register 15 byte coilsValue = mb.Coil(Addr2); // Read Coil input/output </pre>	<pre> // allocate memory in setup mb.addHreg(Addr0, Data); // allocate Holding register + Store Data mb.addIreg(Addr1); // allocate Inputs register mb.addCoil(Addr2); // allocate Coil input/output md.addIsts(Addr3); // allocate Digital inputs // Write data to memory for Client Read mb.Hreg(Addr0, Value); // Write Holding register mb.Iregs(Addr1, Value); // Write Inputs register mb.Coils(Addr2, 1); // Write Coil input/output mb.Ists(Addr3, 1); // Write Digital inputs // From Client Write -> Read data from memory int16_t holdingRegValue = mb.Hreg(Addr0); // Read Holding register byte coilsValue = mb.Coil(Addr2); // Read Coil input/output </pre>
---	---

T00: Blink + Install ModbusIP Library

<pre> 2 #define SW_0 0 3 #define LED_0 2 4 5 void setup() { 6 pinMode(LED_0, OUTPUT); 7 } 8 9 void loop() { 10 digitalWrite(LED_0, HIGH); delay(100); 11 digitalWrite(LED_0, LOW); delay(100); 12 digitalWrite(LED_0, HIGH); delay(100); 13 digitalWrite(LED_0, LOW); delay(800); 14 } 15 </pre>	<pre> #define SW_0 0 #define LED_0 2 void setup() { pinMode(LED_0, OUTPUT); } void loop() { digitalWrite(LED_0, HIGH); delay(100); digitalWrite(LED_0, LOW); delay(100); digitalWrite(LED_0, HIGH); delay(100); digitalWrite(LED_0, LOW); delay(800); } </pre>
<pre> 1 #define SW_0 0 2 #define LED_0 2 3 int sts_Mode = 0; 4 5 void setup() { 6 pinMode(SW_0, INPUT_PULLUP); 7 pinMode(LED_0, OUTPUT); 8 Serial.begin(115200); 9 } 10 11 void loop() { 12 if (digitalRead(SW_0) == 0) { 13 sts_Mode = 1 - sts_Mode; 14 delay(50); 15 Serial.println(sts_Mode); 16 while (digitalRead(SW_0) == 0); 17 delay(50); 18 } 19 if (sts_Mode == 1) { 20 digitalWrite(LED_0, HIGH); delay(100); 21 digitalWrite(LED_0, LOW); delay(100); 22 digitalWrite(LED_0, HIGH); delay(100); 23 digitalWrite(LED_0, LOW); delay(800); 24 } else { 25 digitalWrite(LED_0, HIGH); 26 } 27 } 28 </pre>	<pre> #define SW_0 0 #define LED_0 2 int sts_Mode = 0; void setup() { pinMode(SW_0, INPUT_PULLUP); pinMode(LED_0, OUTPUT); Serial.begin(115200); } void loop() { if (digitalRead(SW_0) == 0) { sts_Mode = 1 - sts_Mode; delay(50); Serial.println(sts_Mode); while (digitalRead(SW_0) == 0); delay(50); } if (sts_Mode == 1) { digitalWrite(LED_0, HIGH); delay(100); digitalWrite(LED_0, LOW); delay(100); digitalWrite(LED_0, HIGH); delay(100); digitalWrite(LED_0, LOW); delay(800); } else { digitalWrite(LED_0, HIGH); } } </pre>

T10: Read/Write Single Coils

```
/*
  Modbus-Arduino Example - Test Led (Modbus IP ESP32)
  Control a Led on GPIO0 pin using Write Single Coil Modbus Function
  Copyright by André Sarmento Barbosa
  http://github.com/andresarmiento/modbus-arduino
*/

#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define mySSID "IoT-Test"
#define myPass "212224236248"

//Modbus Registers Offsets (0-9999)
const int LED_COIL = 100;
//Used Pins
const int ledPin = 2; // LED Test

//ModbusIP object
ModbusIP mb;

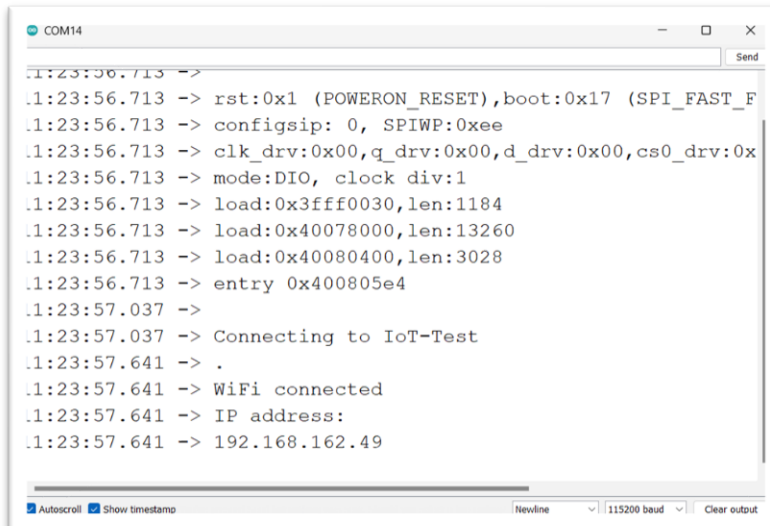
void setup() {
  Serial.begin(115200);
  Serial.print("\nConnecting to ");
  Serial.println(mySSID);
  mb.config(mySSID, myPass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  pinMode(ledPin, OUTPUT);
  mb.addCoil(LED_COIL);
}

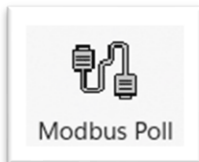
void loop() {
  //Call once inside loop() - all magic here
  mb.task();

  //Attach ledPin to LED_COIL register
  digitalWrite(ledPin, mb.Coil(LED_COIL));
}
```



```
1:23:56.713 ->
1:23:56.713 -> rst:0x1 (POWERON_RESET),boot:0x17 (SPI_FAST_F
1:23:56.713 -> configsip: 0, SPIWP:0xee
1:23:56.713 -> clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x
1:23:56.713 -> mode:DIO, clock div:1
1:23:56.713 -> load:0x3fff0030,len:1184
1:23:56.713 -> load:0x40078000,len:13260
1:23:56.713 -> load:0x40080400,len:3028
1:23:56.713 -> entry 0x400805e4
1:23:57.037 ->
1:23:57.037 -> Connecting to IoT-Test
1:23:57.641 -> .
1:23:57.641 -> WiFi connected
1:23:57.641 -> IP address:
1:23:57.641 -> 192.168.162.49
```

Arduino IDE: Serial
Monitor
>> Connect AP
>> Get IP



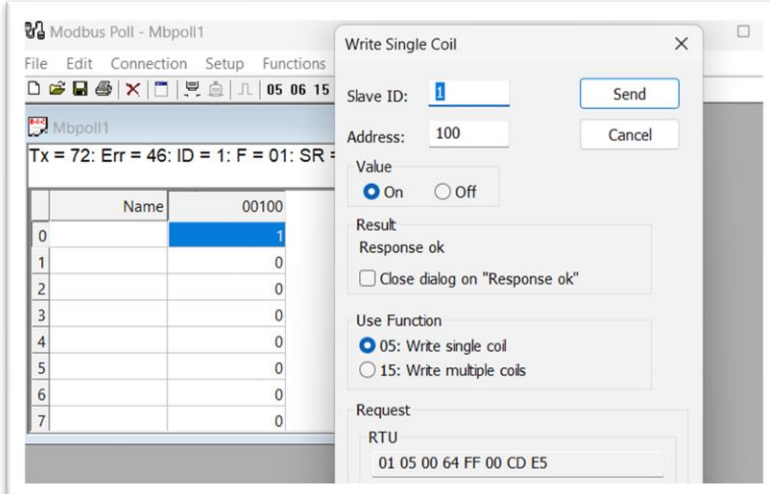
Start Modbus Poll

Modbus Poll – Connection Setup

Modbus TCP/IP
IP Server = xx.xx.xx.xx
Server Port = 502
Timeout = 3000
IP Mode = IPv4

Read/Write Definition

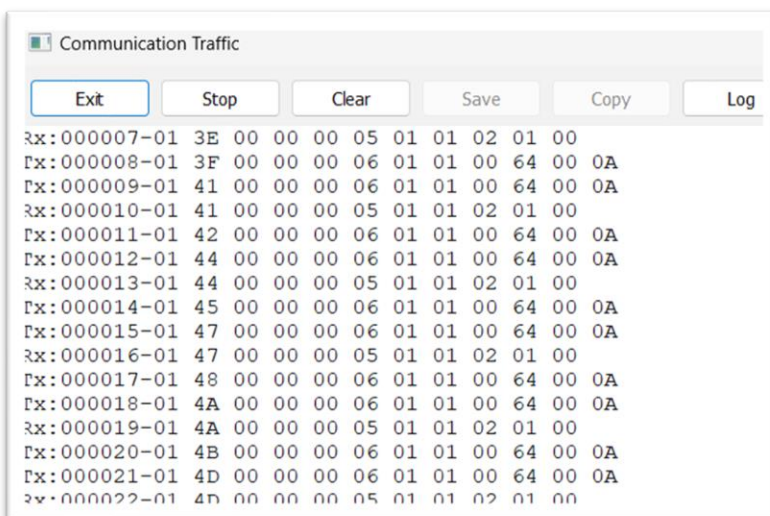
Slave ID = 1
Func = 01 Read Coils
Address Mode = Dec
Address = 100
Quantity = 10
Scan Rate = 1000
View Row = 10



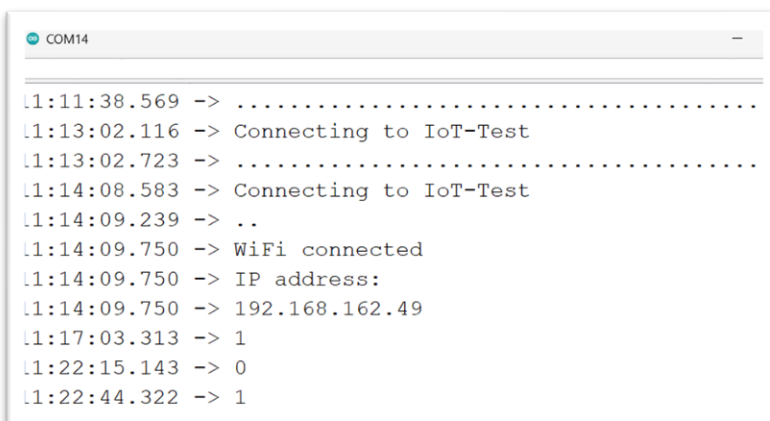
Double Click 100

Slave ID = 1
Address = 100
Value On
Off

Send



Communication Traffic



Arduino IDE: Serial Monitor

T20: Read Holding Register

```
/*
Modbus-Arduino Example - Test Holding Register (Modbus IP ESP32)
Configure Holding Register (offset 100) with initial value 0xABCD
You can get or set this holding register
Copyright by André Sarmiento Barbosa
http://github.com/andresarmiento/modbus-arduino
*/

#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define mySSID "Mue.Home"
#define myPass "myPass1234"

// Modbus Registers Offsets (0-9999)
const int TEST_HREG = 100;

//ModbusIP object
ModbusIP mb;

union unionFloat32Word {
  float myFloat;
  uint16_t myWord[2];
};

union unionFloat64Word {
  double myFloat;
  uint16_t myWord[4];
};

void setup() {
  Serial.begin(115200);
  Serial.print("\n\nConnecting to ");
  Serial.println(mySSID);
  mb.config(mySSID, myPass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.println("-- HEX ASCII --");
  mb.addHreg(TEST_HREG + 0, 0x1234); // 2-Byte ASCII 0000 to FFFF(65535)
  mb.addHreg(TEST_HREG + 1, 0x5678); // 2-Byte ASCII 0000 to FFFF(65535)
  mb.addHreg(TEST_HREG + 2, 0x9ABC); // 2-Byte ASCII 0000 to FFFF(65535)
  mb.addHreg(TEST_HREG + 3, 0xDEF0); // 2-Byte ASCII 0000 to FFFF(65535)

  Serial.println("-- 1Word 2Byte 16Bit Data --");
  int16_t myData_Int = -32765; // 2-Byte Signed -32768 to 32767
  uint16_t myData_uint = 65432; // 2-Byte un-Signed 0 to 65535(2^16 - 1)
  mb.addHreg(TEST_HREG + 4, myData_Int);
  mb.addHreg(TEST_HREG + 5, myData_uint);
  Serial.println(myData_Int);
  Serial.println(myData_uint);

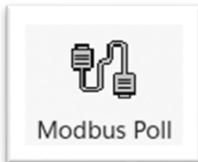
  Serial.println("-- 2Word 4Byte 32Bit Data --");
  long int myData_LgInt = -1234567890; // 4-Byte Signed -2,147,483,648 to 2,147,483,647
  unsigned long int myData_uLgInt = 3210987654; // 4-Byte un-Signed 0 to 4,294,967,295(2^32 - 1)
  unionFloat32Word myFloat32; // 4-Byte float 3.4028235E+38 to -3.4028235E+38
  myFloat32.myFloat = 567.890123456789; // 123.456 >> 0.123456 save 6 to 7 digit
  mb.addHreg(TEST_HREG + 10, myData_LgInt >> 16);
  mb.addHreg(TEST_HREG + 11, myData_uLgInt);
  mb.addHreg(TEST_HREG + 12, myData_uLgInt >> 16);
  mb.addHreg(TEST_HREG + 13, myData_uLgInt);
  mb.addHreg(TEST_HREG + 14, myFloat32.myWord[1]);
  mb.addHreg(TEST_HREG + 15, myFloat32.myWord[0]);
  Serial.println(myData_LgInt);
  Serial.println(myData_uLgInt);
  Serial.println(myFloat32.myFloat, 12);

  Serial.println("-- 4Word 8Byte 64Bit Data --");
  unionFloat64Word myFloat64; // 4-Byte float 3.4028235E+38 to -3.4028235E+38
  myFloat64.myFloat = 567.890123456789; // save 6 to 7 digit
  mb.addHreg(TEST_HREG + 20, myFloat64.myWord[3]);
  mb.addHreg(TEST_HREG + 21, myFloat64.myWord[2]);
  mb.addHreg(TEST_HREG + 22, myFloat64.myWord[1]);
  mb.addHreg(TEST_HREG + 23, myFloat64.myWord[0]);
  Serial.println(myFloat64.myFloat, 12);
}

void loop() {
  //Call once inside loop() - all magic here
  mb.task();
}
```

```
COM14
11:43:44.662 -> Connecting to Mue.Home
11:43:45.268 -> .
11:43:45.268 -> WiFi connected
11:43:45.268 -> IP address:
11:43:45.268 -> 192.168.1.3
11:43:45.268 -> -- HEX ASCII --
11:43:45.268 -> -- 1Word 2Byte 16Bit Data --
11:43:45.268 -> -32765
11:43:45.268 -> 65432
11:43:45.268 -> -- 2Word 4Byte 32Bit Data --
11:43:45.268 -> -1234567890
11:43:45.268 -> 3210987654
11:43:45.268 -> 567.890136718750
11:43:45.268 -> -- 4Word 8Byte 64Bit Data --
11:43:45.268 -> 567.890123456789
```

Serial Monitor



Start Modbus Poll

Connection Setup

Connection: Modbus TCP/IP

Serial Settings: Silicon Labs CP210x USB to UART Bridge (COM), 9600 Baud, 8 Data bits, None Parity, 1 Stop Bit

Mode: RTU (selected), ASCII

Response Timeout: 1000 [ms]

Delay Between Polls: 20 [ms]

Remote Modbus Server: IP Address or Node Name: 192.168.1.3

Server Port: 502, Connect Timeout: 3000 [ms], IPv4 (selected), IPv6

Modbus Poll – Connection Setup

Modbus TCP/IP
IP Server = xx.xx.xx.xx
Server Port = 502
Timeout = 3000
IP Mode = IPv4

Read/Write Definition

Slave ID: OK

Function: 03 Read Holding Registers (4x) Cancel

Address mode
☒ Dec ☐ Hex

Address: PLC address = 40101

Quantity:

Scan Rate: [ms] Apply

Disable
☐ Read/Write Disabled
☐ Disable on error Read/Write Once

View
 Rows
☒ 10 ☐ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity
☐ Hide Name Columns ☐ PLC Addresses (Base 1)
☐ Address in Cell ☐ Enron/Daniel Mode

Request
 RTU 01 03 00 64 00 1E 84 1D

Read/Write Definition

Slave ID = 1
 Func = 03 Read Holding Reg.
 Address Mode = Dec
 Address = 100
 Quantity = 30
 Scan Rate = 1000
 View Row = 10

Mbpoll1

Tx = 3869: Err = 2604: ID = 1: F = 03: SR = 1000ms

Read error

	Name	00100	Name	00110	Name	00120
0		(?) 0x1234		-1234567890		567.890123456789
1		(Vx) 0x5678		Signed	Alt+Shift+S	Format
2		(?) 0x9ABC		Unsigned	Alt+Shift+U	Read/write Definition...
3		(?) 0xDEFO		Hex - ASCII	Alt+Shift+H	Cut
4		-32765		Binary		Copy
5		65432		32 Bit signed		Paste
6		0		32 Bit Unsigned		Select All
7		0		64 Bit Signed		Colors...
8		0		64 Bit Unsigned		Font...
9		0		32 Bit Float		Big-endian
				64 Bit Double		Little-endian

120 → 64Bit Double
 {Format → 64 Bit Double →
 Big-endian}

110 → 32Bit Signed
 112 → 32Bit Unsigned
 114 → 32Bit Float

100 - 103 → ASCII
 104 → Signed
 105 → Unsigned

Mbpoll1

Tx = 3755: Err = 2528: ID = 1: F = 03: SR = 1000ms

Read error

	Name	00100	Name	00110	Name	00120
0		(?) 0x1234		-1234567890		567.890123456789
1		(Vx) 0x5678		--		--
2		(?) 0x9ABC		3210987654		--
3		(?) 0xDEFO		--		--
4		-32765		567.89		0
5		65432		--		0
6		0		--		0
7		0		--		0
8		0		0		0
9		0		0		0

Result Display

T21: Read/Write Holding Register

```
/*
Modbus-Arduino Example - Test Holding Register (Modbus IP ESP32)
Configure Holding Register (offset 100) with initial value 0xABCD
You can get or set this holding register
Copyright by André Sarmiento Barbosa
http://github.com/andresarmiento/modbus-arduino
*/

#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define mySSID "IoT-Test"
#define myPass "pk12345678"

// Modbus Registers Offsets (0-9999)
const int TEST_HREG = 100;

//ModbusIP object
ModbusIP mb;

union unionFloat64Word {
    double myFloat;
    uint16_t myWord[4];
};

unionFloat64Word myFloat64; // 4-Byte float 3.4028235E+38 to -3.4028235E+38
int16_t myData_Int = -32765; // 2-Byte Signed -32768 to 32767
uint16_t myData_uint = 65432; // 2-Byte un-Signed 0 to 65535(2^16 - 1)

void setup() {
    Serial.begin(115200);
    Serial.print("\nConnecting to ");
    Serial.println(mySSID);
    mb.config(mySSID, myPass);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500); Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());

    Serial.println("-- 1Word 2Byte 16Bit Data --");
    mb.addHreg(TEST_HREG + 0, myData_uint); // 2-Byte un-Signed 0 to 65535(2^16 - 1)
    mb.addHreg(TEST_HREG + 1, myData_Int); // 2-Byte Signed -32768 to 32767
    Serial.println(myData_uint);
    Serial.println(myData_Int);

    Serial.println("-- 4Word 8Byte 64Bit Data --");
    myFloat64.myFloat = 567.890123456789; // 4-Byte float 3.4028235E+38 to -3.4028235E+38
    mb.addHreg(TEST_HREG + 6, myFloat64.myWord[3]); // save 6 to 7 digit
    mb.addHreg(TEST_HREG + 7, myFloat64.myWord[2]);
    mb.addHreg(TEST_HREG + 8, myFloat64.myWord[1]);
    mb.addHreg(TEST_HREG + 9, myFloat64.myWord[0]);
    Serial.println(myFloat64.myFloat, 12);
}

void loop() {
    uint16_t returnData;
    unsigned long currentMillis = millis();
    while (millis() - currentMillis < 2500) {
        mb.task();
    }
    returnData = mb.Hreg(TEST_HREG + 0); // return unit16
    Serial.print(returnData, HEX);
    Serial.print("(H)= ");
    Serial.println(returnData);

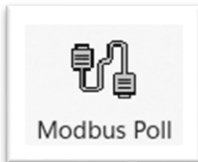
    returnData = mb.Hreg(TEST_HREG + 1); // return unit16
    Serial.print(returnData, HEX);
    Serial.print("(H)= ");
    Serial.println((int16_t)returnData);

    myFloat64.myWord[3] = mb.Hreg(TEST_HREG + 6); // return unit16
    myFloat64.myWord[2] = mb.Hreg(TEST_HREG + 7); // return unit16
    myFloat64.myWord[1] = mb.Hreg(TEST_HREG + 8); // return unit16
    myFloat64.myWord[0] = mb.Hreg(TEST_HREG + 9); // return unit16
    Serial.print(myFloat64.myWord[3], HEX);
    Serial.print(myFloat64.myWord[2], HEX);
    Serial.print(myFloat64.myWord[1], HEX);
    Serial.print(myFloat64.myWord[0], HEX);
    Serial.print("(H)= ");
    Serial.println(myFloat64.myFloat, 12);

    Serial.println();
}
```

```
COM14
5:05:41.674 -> -- 4Word 8Byte 64Bit Data --
5:05:41.674 -> 567.890123456789
5:05:44.147 -> FF98 (H) = 65432
5:05:44.147 -> 8003 (H) = -32765
5:05:44.147 -> 4081BF1EF90C27D (H) = 567.890123456789
5:05:44.147 ->
5:05:46.665 -> FF98 (H) = 65432
5:05:46.665 -> 8003 (H) = -32765
5:05:46.665 -> 4081BF1EF90C27D (H) = 567.890123456789
5:05:46.665 ->
5:05:49.182 -> FF98 (H) = 65432
5:05:49.182 -> 8003 (H) = -32765
5:05:49.182 -> 4081BF1EF90C27D (H) = 567.890123456789
5:05:49.182 ->
```

Arduino IDE: Serial Monitor



Start Modbus Poll
And
Connect Server

Read/Write Definition

Slave ID: OK Cancel

Function: 03 Read Holding Registers (4x) v

Address mode
☒ Dec ☐ Hex

Address: PLC address = 40101

Quantity:

Scan Rate: [ms] Apply

Disable
☐ Read/Write Disabled
☐ Disable on error Read/Write Once

View
Rows
☒ 10 ☐ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity

☐ Hide Name Columns ☐ PLC Addresses (Base 1)
☐ Address in Cell ☐ Enron/Daniel Mode

Request
RTU 01 03 00 64 00 1E 84 1D

Read/Write Definition

Slave ID = 1
Func = 03 Read Holding Reg.
Address Mode = Dec
Address = 100
Quantity = 10
Scan Rate = 1000
View Row = 10

Modbus Poll - [Mbpoll2]

File Edit Connection Setup Functions Display View Window Help

05 06 15 16 17 22 23 TC ? ?

x = 5849: Err = 3984: ID = 1: F = 03: SR = 1000ms

	Name	00100	Name	00110
0		65432		0
1		-32765		0
2		0		0
3		0		0
4		0		0
5		0		0
6		567.890123456789		0
7		--		0
8		--		0
9		--		0

Display

100 → Unsigned (or ASCII)
101 → Signed (or ASCII)

106 → 64Bit Double
{Format → 64 Bit Double → Big-endian}

FF98 (H) = 65432
8003 (H) = -32765
4081BF1EF90C27D (H) = 567.890123456789

Data in HEX format

From >> <https://gregstoll.com/~gregstoll/floattohex/>

Hex value: 0x4081bf1ef90c27d Convert to double

0x4081bf1ef90c27d

4	0	8	1	b	f	1	e	f	9	0	c	0	2	7	d
0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	1

sign exponent mantissa
+1 1032 1.00011011111100011110111100100001100000000100111101 (binary)
+1 * 2^{1032 - 1023} * 1.109160397376541
+1 * 512.000000000000000 * 1.109160397376541
567.890123456789

Double value: 567.890123456789 Convert to hex

Modbus Poll - [Mbpoll2]

File Edit Connection Setup Functions

05 06 15 16 17 22 23 TC ? ?

x = 6301: Err = 4288: ID = 1: F = 03: SR = 1000ms

Read error

	Name	00100
0		43210
1		-32765
2		0
3		0
4		0
5		0
6		567.890123456789
7		--
8		--
9		--

Write Single Register

Slave ID: 1 Send

Address: 100 Cancel

Value: 43210

Result

Response ok

☐ Close dialog on "Response ok"

Use Function

☒ 06: Write single register

☐ 16: Write multiple registers

Request

RTU

01 06 00 64 A8 CA 37 82

Edit

100 = 43210
101 = -4321

Hex value:

0x40092a59b3240022

4				0				0				9				2				a							
0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	1	0	1	0	0	1	0	1	0	0
0	1000000000											1001001010100101100110110011															

sign exponent mantissa
+1 1024 1.100100101010010110011011001
+1 * 2^{^(1024 - 1023)} * 1.572839450615
+1 * 2.0000000000000000 * 1.572839450615

3.14567890123

Double value:

Edit

106 = 3.14567890123

3.14567890123 =
4009 2A59 B324 0022

	Name	00100	Name	00110
0		43210		C
1		-4321		C
2		0		C
3		0		C
4		0		C
5		0		C
6		(@?) 0x4009		C
7		(*Y) 0x2A59		C
8		(?\$) 0xB324		C
9		(?") 0x0022		C

Display 4 Address in ASCII
and Fill

106 = 4009

107 = 2A59

108 = B324

109 = 0022

COM14

```
5:22:02.493 -> EF1F(H) = -4321
5:22:02.493 -> 40092A59B32422(H) = 3.145678901230
5:22:02.493 ->
5:22:05.483 -> A8CA(H) = 43210
5:22:05.483 -> EF1F(H) = -4321
5:22:05.483 -> 40092A59B32422(H) = 3.145678901230
5:22:05.483 ->
5:22:08.570 -> A8CA(H) = 43210
5:22:08.570 -> EF1F(H) = -4321
5:22:08.570 -> 40092A59B32422(H) = 3.145678901230
5:22:08.570 ->
5:22:11.609 -> A8CA(H) = 43210
5:22:11.609 -> EF1F(H) = -4321
5:22:11.609 -> 40092A59B32422(H) = 3.145678901230
5:22:11.609 ->
```

Arduino IDE: Serial Monitor

```
/*
Modbus-Arduino Example - Test Holding Register (Modbus IP ESP32)
Read Analog sensor on Pin ADC (ADC input between 0 ... 1V)
Copyright by André Sarmento Barbosa
http://github.com/andresarmiento/modbus-arduino
*/

#include <WiFi.h>
#include <Modbus.h>
#include <ModbusIP_ESP32.h>

//Modbus Registers Offsets (0-9999)
const int SENSOR_IREG = 100;

//ModbusIP object
ModbusIP mb;

long ts;

void setup() {
  Serial.begin(115200);

  //Config Modbus IP
  mb.config("your_ssid", "your_password");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  // Add SENSOR_IREG register - Use addIreg() for analog Inputs
  mb.addIreg(SENSOR_IREG);

  ts = millis();
}

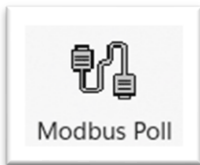
void loop() {
  //Call once inside loop() - all magic here
  mb.task();

  //Read each two seconds
  if (millis() > ts + 2000) {
    ts = millis();
    //Setting raw value (0-1024)
    mb.Ireg(SENSOR_IREG, analogRead(A0));
  }
}
```

COM14

```
15:47:28.746 ->
15:47:28.746 -> Connecting to IoT-Test
15:47:29.354 -> .
15:47:29.354 -> WiFi connected
15:47:29.354 -> IP address:
15:47:29.354 -> 192.168.162.49
15:47:31.831 -> 1-1-1-1-1
15:47:34.350 -> 1-1-1-1-1
15:47:36.831 -> 0-1-1-1-1
15:47:39.361 -> 1-1-1-1-1
```

Arduino IDE: Serial Monitor



Start Modbus Poll
And
Connect Server

Read/Write Definition

Slave ID: 1 OK Cancel

Function: 02 Read Discrete Inputs (1x)

Address mode
☒ Dec ☐ Hex

Address: 100 PLC address = 10101

Quantity: 10

Scan Rate: 1000 [ms] Apply

Disable
☐ Read/Write Disabled
☐ Disable on error Read/Write Once

View
Rows
☒ 10 ☐ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity

☐ Hide Name Columns ☐ PLC Addresses (Base 1)
☐ Address in Cell ☐ Enron/Daniel Mode

Request
RTU 01 02 00 64 00 0A B9 D2

Read/Write Definition

Slave ID = 1
Func = 02 Read Discreet Input
Address Mode = Dec
Address = 100
Quantity = 10
Scan Rate = 1000
View Row = 10

COM14

15:55:58.297 -> 1-1-1-1-1
15:56:00.816 -> 1-1-1-1-1
15:56:03.299 -> 1-1-1-1-1
15:56:05.911 -> 1-1-1-1-1
15:56:08.891 -> 1-1-1-1-1
15:56:11.966 -> 1-1-1-1-1
15:56:14.952 -> 1-1-1-1-1
15:56:18.026 -> 1-1-1-1-1
15:56:20.979 -> 1-1-1-1-1
15:56:23.974 -> 1-1-1-1-1
15:56:27.014 -> 1-1-1-1-1
15:56:30.196 -> 1-0-0-1-1
15:56:33.147 -> 1-0-0-1-1
15:56:36.238 -> 1-0-0-1-1
15:56:39.227 -> 1-0-0-1-1

Modbus Poll - Mbpoll1

File Edit Connection Setup Functions Display View

05 06 15 16 17 22 23 TC

Mbpoll1

Tx = 307: Err = 205: ID = 1: F = 02: SR = 1000ms

Write error

	Name	00100
0		1
1		0
2		0
3		1
4		1
5		0
6		0
7		0
8		0
9		0

For Help, press F1.

Arduino IDE: Serial Monitor
+
Modbus Poll Monitor

T40: Read Input Register

< Input Register -- Read Only >

```
/*
  Modbus-Arduino Example - Test Holding Register (Modbus IP ESP32)
  Read Analog sensor on Pin ADC (ADC input between 0 ... 1V)
  Copyright by André Sarmento Barbosa
  http://github.com/andresarmiento/modbus-arduino
*/

#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define mySSID "IoT-Test"
#define myPass "pk12345678"

//Modbus Registers Offsets (0-9999)
const int SENSOR_IREG = 100;
unsigned long lastMs;
int RandNumber, CountNumber = 0;

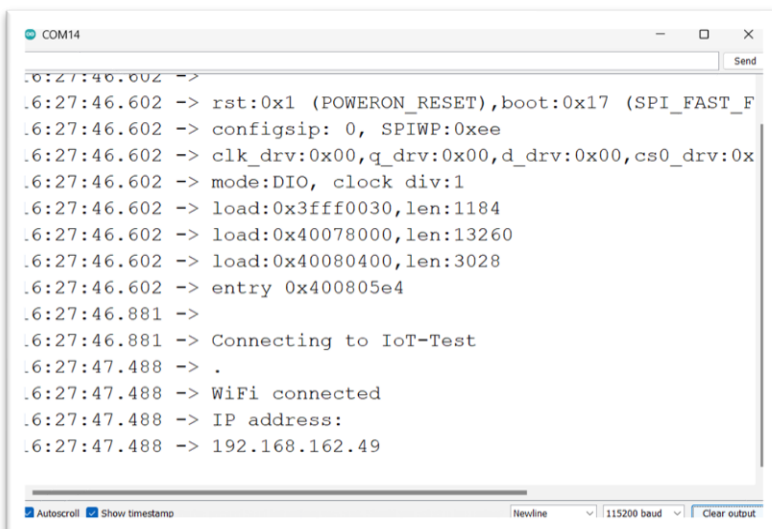
//ModbusIP object
ModbusIP mb;

void setup() {
  Serial.begin(115200);
  Serial.print("\nConnecting to ");
  Serial.println(mySSID);
  mb.config(mySSID, myPass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

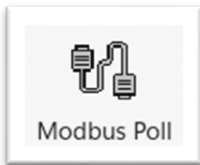
  // Add SENSOR_IREG register - Use addIreg() for test
  mb.addIreg(SENSOR_IREG + 0);
  mb.addIreg(SENSOR_IREG + 1);
  lastMs = millis();
}

void loop() {
  //Call once inside loop() - all magic here
  mb.task();

  //Read each two seconds
  if (millis() - lastMs > 5000) {
    lastMs = millis();
    CountNumber += 1;
    RandNumber = random(1023); // random from 0 to 1023
    mb.Ireg(SENSOR_IREG + 0, CountNumber);
    mb.Ireg(SENSOR_IREG + 1, RandNumber);
    uint16_t returnData = mb.Ireg(SENSOR_IREG + 1);
    Serial.print(CountNumber);
    Serial.print(" >> ");
    Serial.print(RandNumber);
    Serial.print(" - ");
    Serial.println(returnData);
  }
}
```



Arduino IDE: Serial Monitor
Connect Wifi
Get Server IP



Start Modbus Poll
And
Connect Server

Read/Write Definition

Slave ID: OK Cancel

Function: 04 Read Input Registers (3x)

Address mode
☒ Dec ☐ Hex

Address: PLC address = 30101

Quantity:

Scan Rate: [ms] Apply

Disable
☐ Read/Write Disabled
☐ Disable on error Read/Write Once

View
Rows
☒ 10 ☐ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity
☐ Hide Name Columns ☐ PLC Addresses (Base 1)
☐ Address in Cell ☐ Enron/Daniel Mode

Request
RTU 01 04 00 64 00 0A 31 D2
ASCII 3A 30 31 30 34 30 30 36 34 30 30 30 41 38 44 0D 0A

Read/Write Definition

Slave ID = 1
Func = 04 Read Input Register
Address Mode = Dec
Address = 100
Quantity = 10
Scan Rate = 1000
View Row = 10

COM14

16:35:21.982 -> 6 >> 30 -- 30
16:35:26.979 -> 7 >> 60 -- 60
16:35:31.973 -> 8 >> 553 -- 553
16:35:36.960 -> 9 >> 265 -- 265
16:35:41.973 -> 10 >> 602 -- 602
16:35:46.961 -> 11 >> 694 -- 694
16:35:52.186 -> 12 >> 624 -- 624
16:35:57.176 -> 13 >> 488 -- 488
16:36:02.158 -> 14 >> 315 -- 315
16:36:07.332 -> 15 >> 686 -- 686
16:36:12.331 -> 16 >> 746 -- 746
16:36:17.332 -> 17 >> 593 -- 593
16:36:22.414 -> 18 >> 875 -- 875
16:36:27.409 -> 19 >> 565 -- 565
16:36:32.404 -> 20 >> 187 -- 187

Modbus Poll - Mbpoll1

File Edit Connection Setup Functions Display

Mbpoll1
Tx = 630; Err = 429; ID = 1; F = 04; SR = 1000
Read error

	Name	00100
0		19
1		565
2		0
3		0
4		0
5		0
6		0
7		0
8		0
9		0

For Help, press F1.

Autoscroll Show timestamp Newline 115200 baud Clear

Arduino IDE: Serial Monitor
+
Modbus Poll Monitor

T50: Modbus TCP Client Mode

Mission 1/2: Control 4 Relay + Monitor 4 Input

1. ModbusTCP Control 4 Relay with Single Coil
2. ModbusTCP Monitor 4 Input with Discrete Input
3. Control and Display 1+2 On HMI

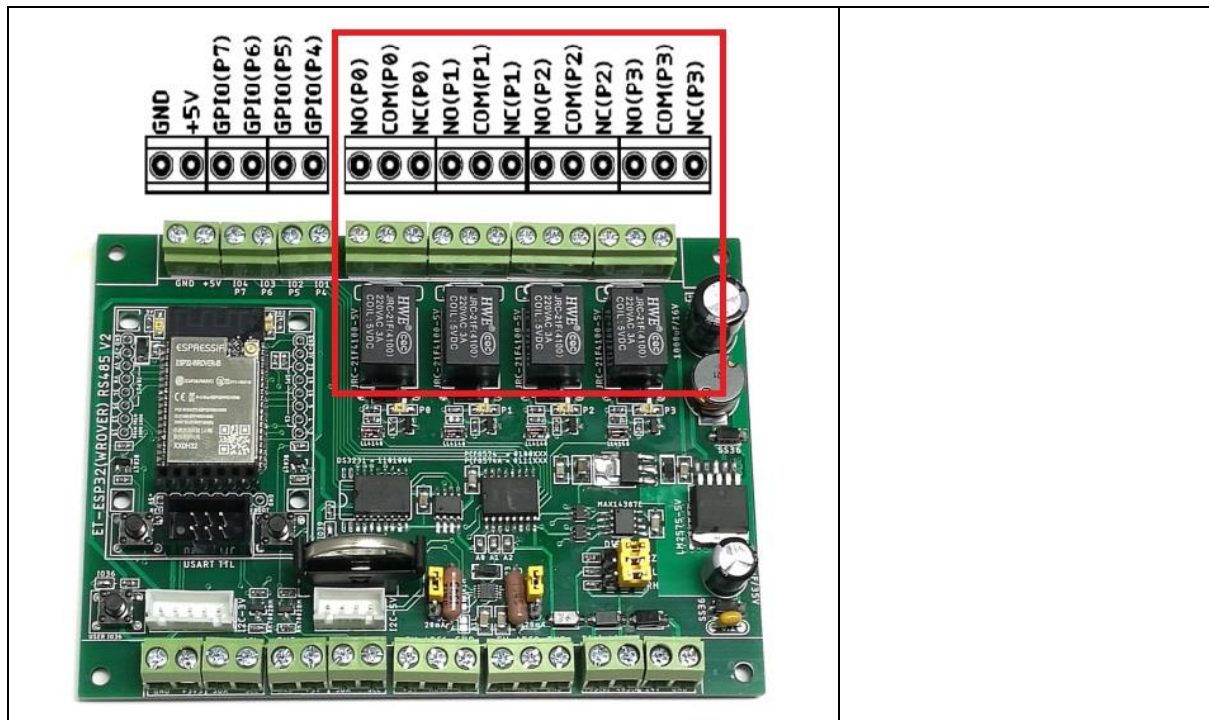
Mission 2/2: Control 4 Relay + Monitor 4 Input + Monitor Analog + Monitor DHT22 (Temp, Humid)

4. ModbusTCP Monitor 2 Analog Input with Input Register in Unsigned format
5. ModbusTCP Monitor ModbusRTU Device with Holding Register in float32 format
6. Control and Display 1+2+4+5 On HMI

Client	Server		
 192.168.1.X	 192.168.1.Z		
 192.168.1.Y			

Mission 1/2: Control 4 Relay + Monitor 4 Input

1. ModbusTCP Control 4 Relay with Single Coil



1.1 Board Test Control Relay → Board = ESP32 WROVER Module

```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library
#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();
}

int pin = 0;
void loop() {
  Serial.println(pin);
  pcf8574.digitalWrite(pin, LOW); delay(500);
  pcf8574.digitalWrite(pin, HIGH); delay(500);
  pin = pin >= 3 ? 0 : pin += 1;
}
```

Relay Control Test

1.2 ModbusIP: Arduino Code for Control Relay by Modbus Coils

```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library

#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

//ModbusIP object
ModbusIP mb;

#define mySSID "Pk.007"
#define myPass "7072126119"

//Modbus Registers Offsets (0-9999)
const int Relay_COIL0 = 100;
const int Relay_COIL1 = 101;
const int Relay_COIL2 = 102;
const int Relay_COIL3 = 103;

int sts_Coil0 = 0;
int sts_Coil1 = 0;
int sts_Coil2 = 0;
int sts_Coil3 = 0;

void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();

  // Relay Output Active Low
  pcf8574.digitalWrite(0, !sts_Coil0);
  pcf8574.digitalWrite(1, !sts_Coil1);
  pcf8574.digitalWrite(2, !sts_Coil2);
  pcf8574.digitalWrite(3, !sts_Coil3);

  Serial.print("\nConnecting to ");
  Serial.println(mySSID);
  mb.config(mySSID, myPass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  mb.addCoil(Relay_COIL0);
  mb.addCoil(Relay_COIL1);
  mb.addCoil(Relay_COIL2);
  mb.addCoil(Relay_COIL3);
}

void display_sts_Coil() {
  Serial.print("Coil Status = ");
  Serial.print(sts_Coil0);
  Serial.print("-");
  Serial.print(sts_Coil1);
  Serial.print("-");
  Serial.print(sts_Coil2);
  Serial.print("-");
  Serial.print(sts_Coil3);
  Serial.println();
}

void loop() {
  mb.task();
  if (mb.Coil(Relay_COIL0) != sts_Coil0) {
    sts_Coil0 = 1 - sts_Coil0;
    pcf8574.digitalWrite(0, !sts_Coil0);
    display_sts_Coil();
  }

  if (mb.Coil(Relay_COIL1) != sts_Coil1) {
    sts_Coil1 = 1 - sts_Coil1;
    pcf8574.digitalWrite(1, !sts_Coil1);
    display_sts_Coil();
  }
}
```

```

if (mb.Coil(Relay_COIL2) != sts_Coil2) {
    sts_Coil2 = 1 - sts_Coil2;
    pcf8574.digitalWrite(2, sts_Coil2);
    display_sts_Coil();
}

if (mb.Coil(Relay_COIL3) != sts_Coil3) {
    sts_Coil3 = 1 - sts_Coil3;
    pcf8574.digitalWrite(3, sts_Coil3);
    display_sts_Coil();
}
}

```

1.3 Test with Modbus Poll

Modbus Poll - Mbpoll1

File Edit Connection Setup Function

05 06

Mbpoll1

Tx = 541: Err = 391: ID = 1: F = 01:

Write error

	Name	00100
0		1
1		1
2		1
3		1
4		0
5		0
6		0
7		0

Write Single Coil

Slave ID: 1

Address: 100

Value

☒ On ☐ Off

Result

N/A

☐ Close dialog on "Response ok"

Use Function

☒ 05: Write single coil

☐ 15: Write multiple coils

Request

RTU

01 05 00 64 FF 00 CD E5

Write Single Coil (05)

Modbus Poll - Mbpoll1

File Edit Connection Setup Functions Display View Window Help

05 06 15 16 17 22 23 TC

Mbpoll1

Tx = 1217: Err = 868: ID = 1: F =

Read error

	Name	00100
0		1
1		0
2		1
3		1
4		0
5		0
6		0
7		0

15: Write Multiple Coils

Slave ID: 1

Address: 100

Quantity: 4

☒ Coil 100

☐ Coil 101

☒ Coil 102

☒ Coil 103

Send

Cancel

Open

Save

Write Multiple Coil (15)

1.4 Test with python

--	--

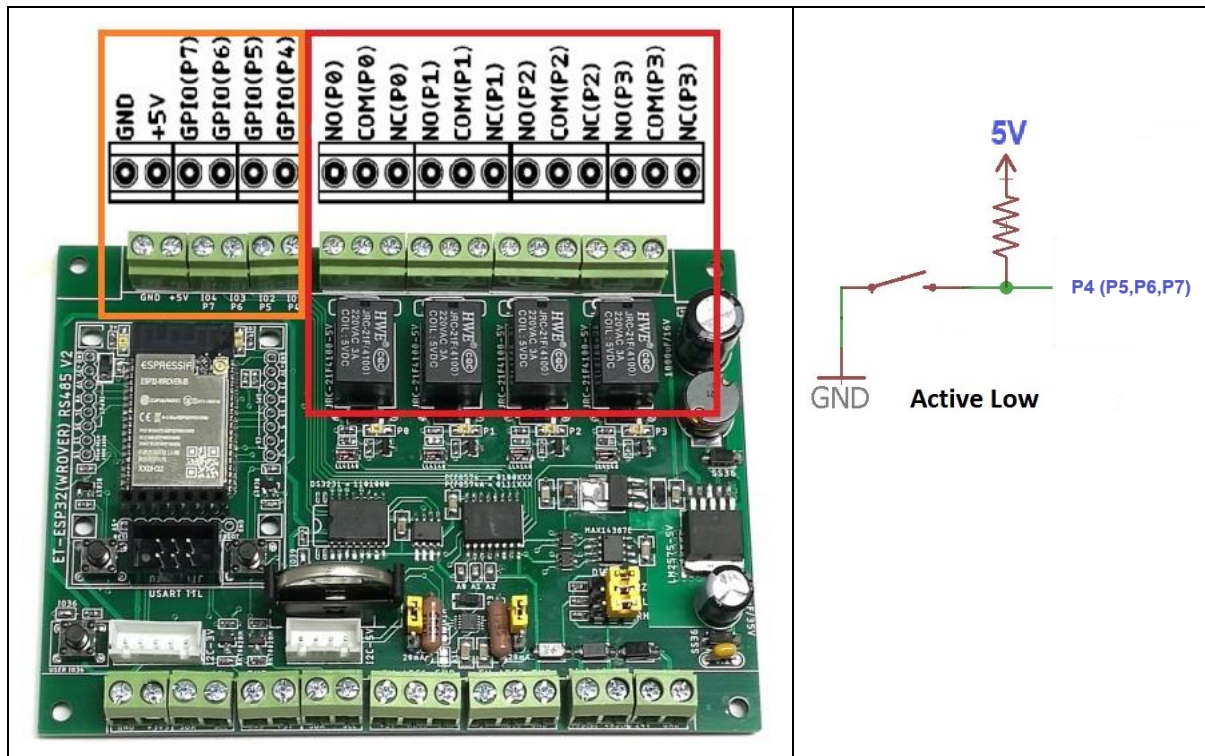
1.5 Test with NodeRED

--	--

1.6 Test with Samkoon HMI

--	--

2. ModbusTCP Monitor 4 Input with Discreet Input



2.1 Board Test Read Digital Input → Board = ESP32 WROVER Module

```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library
#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();
}

int pin = 0;
void loop() {
  uint8_t val = pcf8574.digitalRead(P4);
  if (val == LOW) Serial.println("KEY PRESSED");
  delay(50);
}
```

Input Test

2.2 ModbusIP Code for Read Digital Input and store in Modbus Discreet Input

--	--

2.3 Test with Modbus Poll

--	--

2.4 Test with python

--	--

2.5 Test with NodeRED

--	--

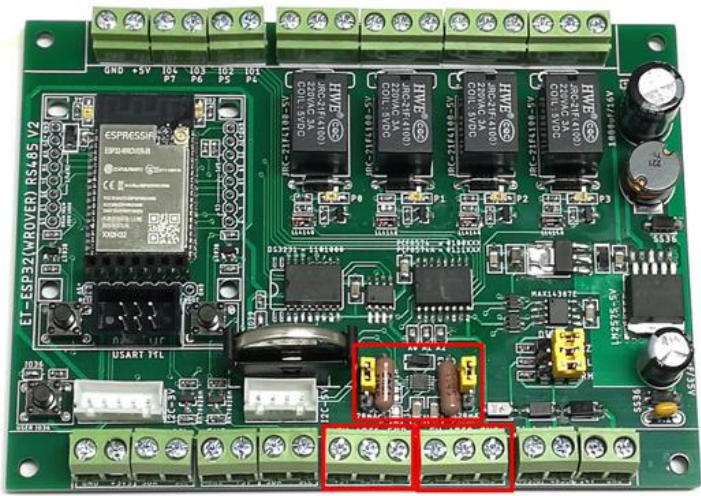
2.6 Test with Samkoon HMI

--	--

3. Control and Display 1+2 On HMI

Mission 2/2: Control 4 Relay + Monitor 4 Input + Monitor Analog + Monitor DHT22 (Tempp, Humid)

4. ModbusTCP Monitor 2 Analog Input with Input Register in Unsigned format





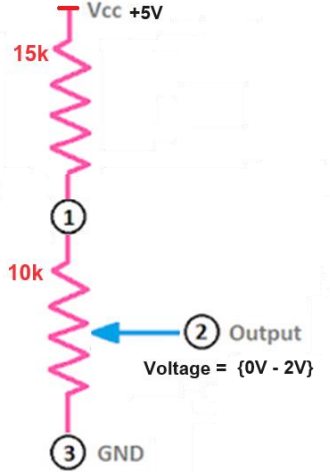
+5V
ADC1
GND

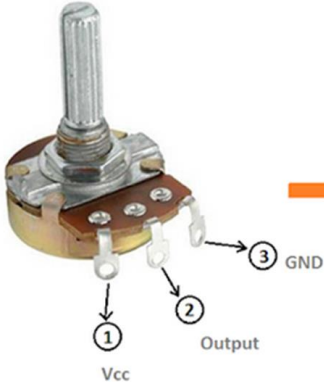


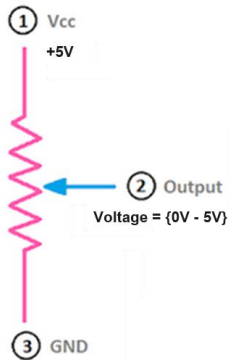
+5V
ADC2
GND

ADC1, ADC2

Input Range 0.0 - 2.0 V

$$V_1 = 10k \frac{5V}{(10k + 15k)} = 2V$$






4.1 Board Test for Read Analog Input ➔ Board = ESP32 WROVER Module

	Add Lib: mcp3423 Ver 1.0.4
<pre>#include <Wire.h> #include <MCP342x.h> #define I2C_Address 0x6D #define I2C_SDA_Pin 21 #define I2C_SCL_Pin 22 MCP342x adc = MCP342x(I2C_Address); void setup(void) { Serial.begin(115200); Wire.begin(I2C_SDA_Pin, I2C_SCL_Pin); // Reset devices MCP342x::generalCallReset(); delay(1); // MCP342x needs 300us to settle, wait 1ms // Check device present Wire.requestFrom(I2C_Address, (uint8_t)1); if (!Wire.available()) { Serial.print("No device found at address "); Serial.println(I2C_Address, HEX); while (1); } } void loop(void) { long Value_01, Value_02; MCP342x::Config status; // Initiate a conversion; convertAndRead() will wait until it can be read delay(250); adc.convertAndRead(MCP342x::channel1, MCP342x::oneShot, MCP342x::resolution16, MCP342x::gain1, 1000000, Value_01, status); delay(250); adc.convertAndRead(MCP342x::channel2, MCP342x::oneShot, MCP342x::resolution16, MCP342x::gain1, 1000000, Value_02, status); Serial.print("\n Value 1,2 : "); Serial.print(Value_01); Serial.print(", "); Serial.print(Value_02); }</pre>	

4.2 ModbusIP Code for Read Analog Input and store data to Holding Register in Unsigned format

```
#include <Wire.h>
#include <MCP342x.h>
#include <Modbus.h>
#include <ModbusIP_ESP32.h>

#define I2C_Address 0x6D
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

#define mySSID "Pk.007"
#define myPass "7072126119"

//Modbus Registers Offsets (0-9999)
const int SENSOR_IREG = 100;
unsigned long lastMs;
long Value_01, Value_02;
int ADC_Select = 0;

MCP342x adc = MCP342x(I2C_Address);
ModbusIP mb;

void setup(void) {
  lastMs = millis();
  Serial.begin(115200);
  Serial.print("\nConnecting to ");
  Serial.println(mySSID);
  mb.config(mySSID, myPass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  // Add SENSOR_IREG register - Use addIreg() for test
  mb.addIreg(SENSOR_IREG + 0);
  mb.addIreg(SENSOR_IREG + 1);

  Wire.begin(I2C_SDA_Pin, I2C_SCL_Pin);

  // Reset devices
  MCP342x::generalCallReset();
  delay(1); // MCP342x needs 300us to settle, wait 1ms

  // Check device present
  Wire.requestFrom(I2C_Address, (uint8_t)1);
  if (!Wire.available()) {
    Serial.print("No device found at address ");
    Serial.println(I2C_Address, HEX);
    while (1);
  }
}

void loop(void) {
  //Call once inside loop() - all magic here
  mb.task();

  //Read each two seconds
  if (millis() - lastMs > 5000) {
    lastMs = millis();
    ADC_Select = 1 - ADC_Select;
    MCP342x::Config status;
    if (ADC_Select == 1) {
      adc.convertAndRead(MCP342x::channel1, MCP342x::oneShot,
        MCP342x::resolution16, MCP342x::gain1,
        1000000, Value_01, status);
      mb.Ireg(SENSOR_IREG + 0, Value_01);
    }
    else {
      adc.convertAndRead(MCP342x::channel2, MCP342x::oneShot,
        MCP342x::resolution16, MCP342x::gain1,
        1000000, Value_02, status);
      mb.Ireg(SENSOR_IREG + 1, Value_02);
    }

    Serial.print("\n Value 1,2 : ");
    Serial.print(Value_01);
    Serial.print(", ");
    Serial.print(Value_02);
  }
}
```

4.3 Test with Modbus Poll

16:36:04.000 -> Value 1,2 : 13308, 19370

16:36:08.993 -> Value 1,2 : 31384, 14587

16:36:14.021 -> Value 1,2 : 0, 0

16:36:19.929 -> Value 1,2 : 0, 0

16:36:24.959 -> Value 1,2 : 0, 0

16:36:29.932 -> Value 1,2 : 0, 0

16:36:35.107 -> Value 1,2 : 0, 0

16:36:40.082 -> Value 1,2 : 0, 0

16:36:45.115 -> Value 1,2 : 0, 0

16:36:50.282 -> Value 1,2 : 0, 0

16:36:55.265 -> Value 1,2 : 0, 0

16:37:00.247 -> Value 1,2 : 0, 0

16:37:05.415 -> Value 1,2 : 0, 0

16:37:11.329 -> Value 1,2 : 0, 0

16:37:16.360 -> Value 1,2 : 31384, 14587

Modbus Poll - Mbpoll1

File Edit Connection Setup Functions Display View Window Help

Tx = 64: Err = 42: ID = 1: F = 04: SR = 1000ms

	Name	00100
0		31384
1		14587
2		0
3		0
4		0
5		0
6		0
7		0

For Help, press F1. [192.168.56.53]

☒ Autoscroll ☒ Show timestamp Newline 115200 baud

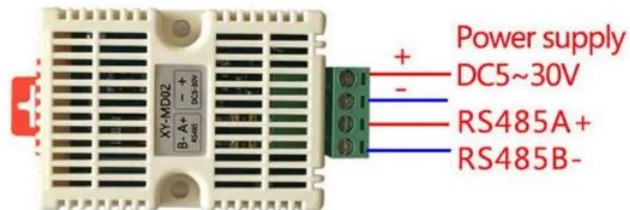
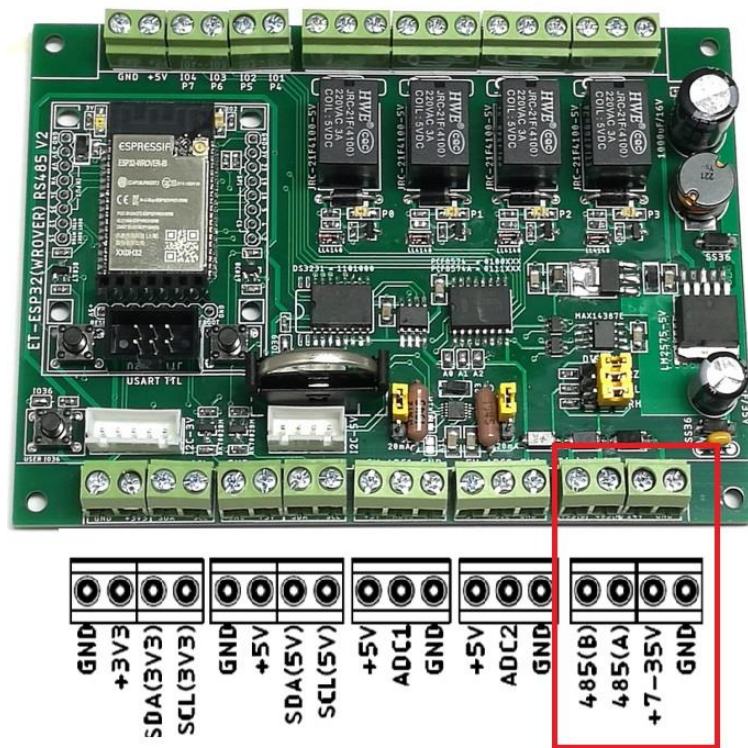
Read Input Register
Addr = 100
Addr = 101

4.4 Test with python

4.5 Test with NodeRED

4.6 Test with Samkoon HMI

5. ModbusTCP Monitor MocbusRTU Device with Holding Register in float32 format



5.1 Board Test for Read XY-MD02 via modbusRTU → Board = ESP32 WROVER Module

--	--

5.2 ModbusIP Code for Read modbusRTU and store data to Holding Register in float32 format

--	--

5.3 Test with Modbus Poll

--	--

5.4 Test with python

--	--

5.5 Test with NodeRED

--	--

5.6 Test with Samkoon HMI

--	--

6. Display 1+2+4+5 On HMI
